

CORPUS DB PDF EDITION

Database Documentation

Schema, evidence, materialization,
ontology, and operational read model.

Core rule: the Artifact Tree explains how evidence moved through the pipeline; the Corpus DB explains the current materialized state.

Prepared from `07_Database_Documentation.md`. Revised for readability, clean tables, stable sectioning, and rendered operational diagrams.

Contents

1. Corpus DB mental model	3
2. Source Of Truth	4
3. DB File Locations	5
4. Schema Creation Flow	6
5. Table Families	7
6. Read-Surface Views	13
7. Materialization Lifecycle	16
8. Source Document Grouping And Base Graph	17
9. Ontology Layer	18
10. Embeddings And Search	19
11. Page Images And Source Viewer	20
12. Query Agent Read Model	21
13. Frontend DB Status	23
14. Safe External Inspection	24
15. Mutation Rules	25
16. Reset, Cleanup And Merge	26
17. Debugging Checklist	27
18. Test Anchors	28
19. Handover Rule	29

Recommended first pass: read sections 1-6, then jump to the operational section that matches the task at hand.

1. Corpus DB mental model

The Corpus DB is the queryable materialization of an Artifact Tree. It is the surface the Query Agent reads, the Ontology Agent extends, the Frontend viewer uses for source evidence, and the Corpus Builder owns for schema creation, materialization, rebuild, reset and merge.

The Artifact Tree explains how evidence moved through the pipeline. The Corpus DB explains the current materialized state of that evidence.

Both are needed for real debugging. A DB row without the Artifact Tree can be queried, but it is harder to audit. An Artifact Tree without the DB can be inspected, but it is not the operational retrieval surface.

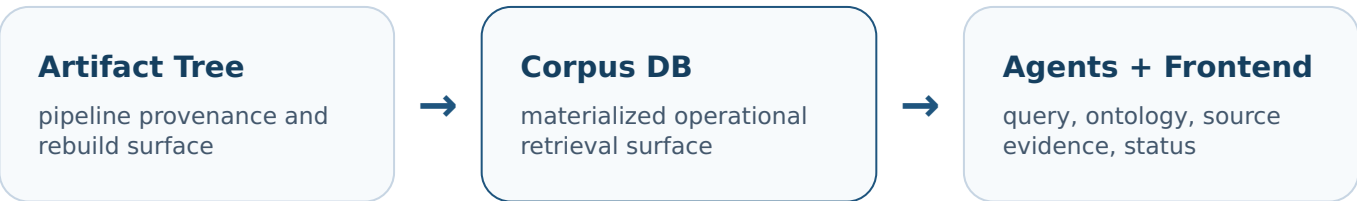
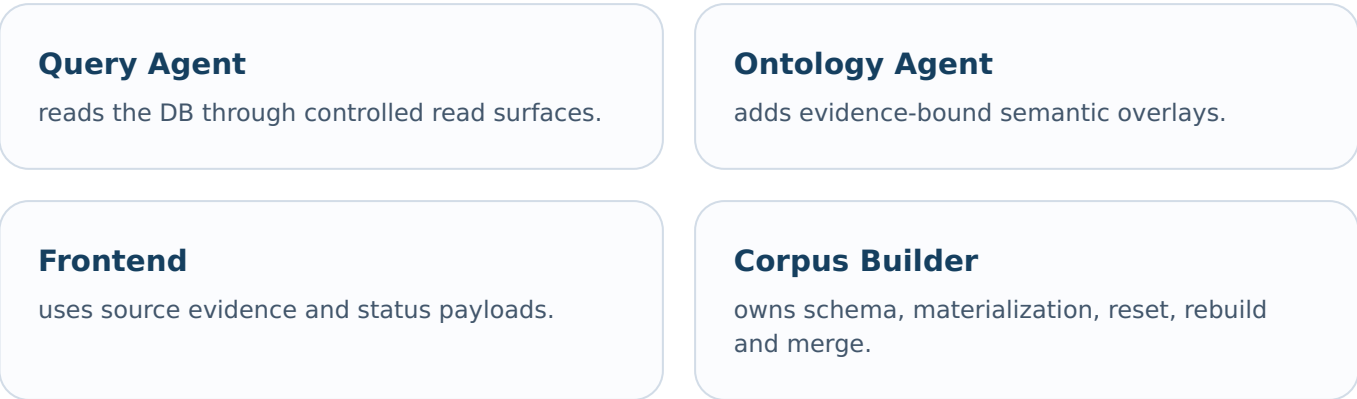


Figure 1. The Artifact Tree explains how evidence moved through the pipeline; the Corpus DB explains the current materialized state.



2. Source Of Truth

The active schema contract lives in:

- 05 - Corpus Builder/corpus_builder/database/types.py
- 05 - Corpus Builder/corpus_builder/database/workflow.py
- 05 - Corpus Builder/corpus_builder/database/schema_core.py
- 05 - Corpus Builder/corpus_builder/database/schema_semantics.py
- 05 - Corpus Builder/corpus_builder/database/validation.py

The current Corpus DB schema version is:

10

The schema is modular. `schema_core.py` aggregates document, page image, search, ontology and structural tables. `schema_semantics.py` adds evidence and materialization tables. `workflow.ensure_schema()` creates tables, indexes, FTS and read-surface views, then seeds `installation_state`.

Current contract size:

Item	Count
Physical tables	41
Explicit indexes	129
Read-surface views	20
FTS5 virtual tables	1
Triggers	0

Older DBs are not blindly upgraded. The compatibility validator allows the current schema and a narrow previous runtime version, but rejects legacy Semantic Bundle schemas and old taxonomy/mapping columns that would make the DB semantically unsafe. In those cases the correct path is rebuild, not silent migration.

3. DB File Locations

Normal production DBs live under an Artifact Tree:

```
Artifact Tree/ Corpus/ corpus.db corpus.db-wal corpus.db-shm
```

Kernel workflows expect the selected DB to stay inside the selected Artifact Tree's **Corpus** folder. That is part of the target identity model. A DB outside the artifact root may still be a valid SQLite file, but it is not a valid Kernel-controlled target for ingest, reset, rebuild or merge.

The Corpus Builder module config has its own fallback path for module-local runs, but handover and product workflows should use the Artifact Tree layout. The Client Frontend stores the active Query Agent database path in its config and opens it read-only for Query Agent work.

SQLite sidecars:

- ***.db-wal** is the write-ahead log.
- ***.db-shm** is the shared-memory sidecar.

Do not delete sidecars while a process owns the DB. They may be cleaned only when the DB is closed and the workflow explicitly does checkpoint/cleanup.

4. Schema Creation Flow

`ensure_schema()` is the schema gate.

validate

→

create tables

→

indexes + FTS

→

views

→

seed state

Figure 2. Schema setup is an owner-controlled gate, not a loose SQL script.

It does this in order:

1. Validate existing DB compatibility.
2. Drop deprecated tables `document_slot_candidates` and `document_slots`.
3. Create contract tables.
4. Run additive runtime migrations.
5. Create indexes.
6. Create the `documents_fts` FTS5 virtual table.
7. Drop and recreate read-surface views.
8. Backfill source identity for older compatible rows.
9. Seed `installation_state`.
10. Commit.

The important point is that schema setup is owner code, not a loose SQL file operators are expected to run manually.

5. Table Families

Core Document Tables

Table	Purpose
<code>documents</code>	One materialized page/unit record with source identity, classification, review state, projection state and content summary fields.
<code>document_payloads</code>	Cold payload storage for structured, raw, normalized, projection JSON and original file blob metadata.
<code>extracted_fields</code>	Flattened scalar values from <code>content.fields</code> .
<code>extracted_rows</code>	Table/list rows as JSON.
<code>relations</code>	Document-level relations and Base Graph page relations.
<code>tags</code>	Tags with normalized/compact values.
<code>people</code>	Person names with normalized/compact values.
<code>organizations</code>	Organization names with normalized/compact values.

`documents` is still page-level materialization. It now carries explicit source identity fields such as `source_document_id`, `source_uri`, `page_index`, `page_label`, source hashes and ingest IDs. The Base Graph later uses those fields to build document-level grouping.

Do not treat `documents.page_count` as "total pages in the corpus". It is a per-row/page-context field. Use `source_document_pages` or `structural_units` when the question is about actual source-document page coverage.

Evidence And Promotion Tables

Table	Purpose
<code>evidence_atoms</code>	Atomic evidence with JSON path, page/row anchors, normalized text, numeric/date forms and source refs.
<code>slot_candidates</code>	Candidate values for dynamic slots, including ambiguity and projection backing.
<code>document_promotions</code>	Chosen/current promoted values that become convenient query fields.
<code>candidate_evidence</code>	Link table between candidates and concrete evidence atoms.

This family is what makes the DB usable beyond a flat JSON dump. It preserves candidate ambiguity, final promotions and the evidence atoms behind them.

`document_promotions` is especially important for dynamic taxonomies. It is where projection-backed values become compact, indexed, query-friendly facts without hardcoding a fixed business schema into the DB.

Materialization And Release State

Table	Purpose
<code>installation_state</code>	Singleton row for current schema and active Semantic Release identity.
<code>semantic_snapshots</code>	Embedded runtime truth for activated Semantic Releases.
<code>document_processing_state</code>	Per-document materialization snapshot, projection and stale state.
<code>document_entities</code>	Materialized semantic entities.
<code>entity_attributes</code>	Attributes belonging to materialized entities.
<code>entity_relations</code>	Semantic relations between entities and optional document targets.
<code>semantic_evidence_links</code>	Links semantic subjects to <code>evidence_atoms</code> .
<code>materialization_runs</code>	Audit records for materialization/backfill/release runs.
<code>materialization_audit</code>	Warnings/errors/details from materialization.

The active release snapshot is the DB-local semantic truth. A Semantic Release file in the Artifact Tree is the filesystem-side package. The DB snapshot is what the materialized rows were bound to at runtime.

Page Image Table

Table	Purpose
<code>document_page_images</code>	DB-local page image BLOBs keyed by <code>(document_id, page)</code> .

For V1 databases, this table is part of the expected evidence model. It lets the Frontend source viewer and DB-local evidence backlinking retrieve visual page evidence without relying only on filesystem lookup.

The Artifact Tree still keeps `Documents/page_images` as the artifact-level ground truth and rebuild surface. The DB-local BLOB is the query/viewer-local copy.

The Frontend image resolver prefers DB BLOBs and falls back to `Documents/page_images` files.

Search And Embedding Tables

Table	Purpose
<code>embedding_chunks</code>	Chunk-level vectors for document segments, rows, fields and page-level text.
<code>embeddings</code>	Document-level curated embedding text and vector.
<code>documents_fts_content</code>	Backing content table for full-text search.
<code>documents_fts</code>	FTS5 virtual table over free text, fields, tags, people and organizations.
<code>load_history</code>	Load, skip, archive and hash-change history.

`embedding_chunks` is preferred for modern semantic search. `embeddings` is the older/coarser document-level surface. Query Agent semantic search tries vector search when vectors exist and falls back to lexical search when embeddings are unavailable.

When using raw SQLite manually, `documents_fts_content` is backing storage. `MATCH` queries should target `documents_fts`.

Source Document And Base Graph Tables

Table	Purpose
source_documents	Aggregates page-level documents rows into source documents.
source_document_pages	Maps each source document to ordered page-level document rows.
source_document_classifications	Source-level classification in base , semantic_release or ontology scope.
structural_units	Deterministic segment units such as base_unit , page_unit , future chapter , section , page_span .
structural_unit_relations	contains , next and previous links between structural units.
relations	Also stores Base Graph document/page relations with relation_origin='base_graph' .

Base Graph data is deterministic. basic_relation_mining does not ask an LLM to decide what a document means. It groups by explicit source_document_id , rejects missing identity and duplicate page indexes, writes page order and structural units, then reports unresolved ambiguity instead of inventing structure.

The initial structural layer creates base_unit and page_unit . The schema already allows chapter , section and page_span , but those are target shapes for future deterministic or ontology-driven segmentation work.

Ontology Tables

Table	Purpose
<code>ontology_lenses</code>	User-selectable knowledge lenses over the same corpus.
<code>ontology_runs</code>	Checkpointable ontology run metadata.
<code>ontology_terms</code>	Lens-local terminology.
<code>ontology_nodes</code>	Lens-local graph nodes.
<code>ontology_edges</code>	Lens-local graph edges.
<code>ontology_assertions</code>	Subject-predicate-object/value assertions.
<code>ontology_evidence_links</code>	Evidence links from ontology objects to corpus evidence.
<code>ontology_activation</code>	Active/primary lens selection for the corpus.
<code>ontology_embedding_chunks</code>	Vectors for ontology objects.
<code>ontology_edit_log</code>	SQL edit units, verification state and before/after metadata.

Ontology tables are separate from the base `relations` table. `relations` is for the corpus Base Graph and document-level relations. Ontology lenses are versioned semantic overlays that may interpret, critique, correct, review or reframe the same materialized base without overwriting it.

Important status values:

Object	Status Values
<code>ontology_lenses.status</code>	<code>draft</code> , <code>ready</code> , <code>archived</code>
<code>ontology_lenses.embedding_status</code>	<code>dirty</code> , <code>pending</code> , <code>clean</code> , <code>failed</code> , <code>unavailable</code>
<code>ontology_runs.status</code>	<code>running</code> , <code>waiting_user</code> , <code>complete</code> , <code>failed</code> , <code>rolled_back</code>
terms	<code>draft</code> , <code>verified</code> , <code>rejected</code> , <code>deprecated</code>
nodes	<code>draft</code> , <code>proposed</code> , <code>verified</code> , <code>rejected</code> , <code>deprecated</code>
edges/assertions	<code>draft</code> , <code>proposed</code> , <code>verified</code> , <code>rejected</code> , <code>deprecated</code> , <code>hypothesis</code>

There is exactly one active primary corpus lens because `ontology_activation` has a unique active primary constraint.

6. Read-Surface Views

Read-surface views are recreated during `ensure_schema()`. They are the safer surface for agents and maintainers because they hide some join complexity and present source documents, promotions, structural units and active ontology in a more queryable shape.

Base Views

View	Purpose
<code>vw_base_evidence_atoms</code>	Direct evidence atom read surface.
<code>vw_base_slot_candidates</code>	Base-layer slot candidates only.
<code>vw_document_promotions_current</code>	Current promoted document values.
<code>vw_document_header_surface</code>	Document headers joined with promoted values.
<code>vw_document_search_surface</code>	Document-level text surface for search.
<code>vw_observed_semantics</code>	Observed entity/attribute/relation semantics.
<code>vw_materialized_semantics</code>	Materialized entity/attribute/relation semantics.

Source And Structure Views

View	Purpose
<code>vw_source_document_pages</code>	Ordered page rows for each source document.
<code>vw_source_document_classifications</code>	Source-level classifications by scope.
<code>vw_source_document_surface</code>	Summary surface for source documents.
<code>vw_same_source_document_pages</code>	Page-pair helper for same-source joins.
<code>vw_structural_units</code>	Structural units with source/document context.
<code>vw_structural_unit_relations</code>	Structural unit relation surface.
<code>vw_source_document_entities</code>	Entities projected to source-document page context.
<code>vw_source_document_evidence_atoms</code>	Evidence atoms projected to source-document page context.
<code>vw_source_document_promotions</code>	Current promotions projected to source-document page context.

Ontology Views

View	Purpose
<code>vw_active_ontology_nodes</code>	Nodes from the active primary ready lens.
<code>vw_active_ontology_edges</code>	Edges from the active primary ready lens with evidence counts.
<code>vw_active_ontology_assertions</code>	Assertions from the active primary ready lens with evidence counts.
<code>vw_query_surface_with_active_ontology</code>	Unified query surface across source documents, structure and active ontology.

For direct inspection, start with these views before writing deep joins by hand:

- `vw_source_document_surface`
- `vw_source_document_pages`
- `vw_document_promotions_current`
- `vw_document_search_surface`
- `vw_structural_units`

- `vw_query_surface_with_active_ontology`

7. Materialization Lifecycle

The normal load path is:

```
normalized artifact -> load_batch -> ensure_schema -> ensure active Semantic Release ->  
load_from_file -> load_document transaction -> semantic release domain materialization ->  
DB writes -> FTS/history/page image/embedding-ready state
```

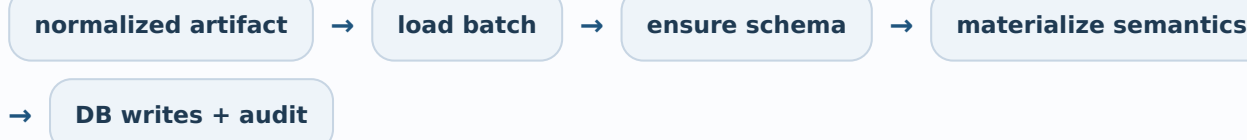


Figure 3. Normal ingest moves from normalized artifacts into evidence-bound DB state.

`load_document` is the main transaction owner. It prepares the bundle, materializes semantic-release output, builds the document header, writes core document tables, writes payloads, writes page images if configured, writes evidence/candidates/promotions, writes semantic entities and relations, updates FTS content, records load history, then commits.

The semantic release materializer does domain work before DB writes:

- validate projection binding
- produce promoted slots
- produce entity/attribute/relation payloads
- produce processing state and audit payloads

This prevents the database layer from becoming a pile of prompt-shaped exceptions.

8. Source Document Grouping And Base Graph

The ingestion pipeline materializes page-level records first. That is intentional: the page image is the inspectable evidence unit. Source-document grouping is then made explicit by Base Graph mining.

`basic_relation_mining` writes:

- `source_documents`
- `source_document_pages`
- `source_document_classifications`
- Base Graph `relations`
- `structural_units`
- `structural_unit_relations`

It clears and rewrites the deterministic Base Graph layer before committing. If validation fails, the transaction rolls back.

The classification rule is conservative:

- if all pages agree, source-level classification can be `materialized`
- if pages disagree, it becomes `ambiguous`
- if there is not enough evidence, it becomes `unresolved`

The table supports three classification scopes:

Scope	Writer
<code>base</code>	Deterministic Base Graph / page consensus
<code>semantic_release</code>	Deterministic projection/release-derived consensus
<code>ontology</code>	Ontology lens-specific interpretation

The Ontology Agent is blocked from writing `base` or `semantic_release` classifications through its normal preflight. Those are owner-controlled layers.

9. Ontology Layer

The ontology layer is a DB-native knowledge mining layer.

An ontology lens can represent a reading, review, critique, correction, semantic map or domain-specific interpretation over the same materialized corpus. It does not overwrite base facts. It adds evidence-bound objects that the Query Agent can read alongside the base corpus.

Evidence links are explicit. `ontology_evidence_links` can point ontology objects to:

- `document`
- `source_document`
- `structural_unit`
- `evidence_atom`
- `promotion`
- `field`
- `row`

Ontology writes go through the Ontology Agent's `sql_batch_execute` tool. The tool is atomic: it normalizes/allowlists statements, preflights references, opens `BEGIN IMMEDIATE`, runs the batch, marks affected lenses dirty, commits, runs Kernel validation, refreshes ontology embeddings if possible, and records an `ontology_edit_log` entry.

Common write-shape failures that the tool/preflight must catch:

- missing stable IDs
- missing `attributes_json`
- invalid lens status such as `active`
- edge endpoints that are not nodes
- cross-lens edge endpoints
- evidence links to objects that do not exist
- writes to base or semantic-release classifications
- ontology embedding chunks without `object_id`

10. Embeddings And Search

There are three related search layers:

Layer	Surface
FTS	<code>documents_fts</code> over text fields and metadata.
Document vectors	<code>embeddings</code> .
Chunk vectors	<code>embedding_chunks</code> .

The Query Agent prefers chunk vectors when available, can fall back to document-level vectors, and can fall back to lexical search if embeddings are unavailable. Missing embeddings reduce semantic search quality, but they do not make the DB invalid as a materialized corpus.

Ontology embeddings are separate in `ontology_embedding_chunks`. They are refreshed after valid ontology edits when credentials exist. If no provider is configured, lens `embedding_status` can become `unavailable`; the lens itself can still be usable through SQL reads.

11. Page Images And Source Viewer

The DB-local page image contract is:

```
document_page_images( document_id, page, content_type, byte_size, image_sha256,  
image_blob )
```

The Frontend source viewer resolves images DB-first and filesystem-fallback. The HTTP endpoint serves:

```
GET /api/image/<docId>/<page>
```

Use this mental model:

- `Documents/page_images` is artifact-level visual ground truth and rebuild evidence.
- `document_page_images` is DB-local evidence backlinking and viewer material.

If either side is missing, note the integrity gap. The system may still partially work, but audit/rebuild/viewing are not all equally complete.

12. Query Agent Read Model

The Query Agent opens the configured DB read-only. Its SQL tool accepts only a single `SELECT` or `WITH` statement and blocks mutating keywords.

Primary read tools:

- `sql_query`
- `get_document_summary`
- `get_document_ontology_evidence`
- `get_document_rows`
- `get_document_provenance`
- `get_document_full`
- `get_document`
- `get_provenance`
- `semantic_search`
- `database_coverage_snapshot`
- `list_source_documents`
- `get_source_document`
- `list_ontology_lenses`
- `get_ontology_lens`
- `workbench`

The document tools are layered views over the same repository-owned DB reader:

Tool	Use It For
<code>get_document_summary</code>	document identity, source-document context, active promotions, structural hints and short excerpts
<code>get_document_ontology_evidence</code>	lens work, source-document classifications, selected fields/rows, structural units, evidence atoms and bounded payload excerpts
<code>get_document_rows</code>	row-heavy material such as invoice lines, order lines, shipment rows or table-like extracted content
<code>get_document_provenance</code>	document-level provenance material before the exact target slot is known
<code>get_document_full</code> / <code>get_document</code>	full document inspection when compact views are insufficient

This is a token-control and field-hardening surface, not a second schema. The underlying DB truth remains `documents`, payloads, promotions, fields, rows, evidence atoms, source-document tables, structural units and ontology tables.

`database_coverage_snapshot` is an interactive agent coverage view. It is not the same thing as Kernel support evidence. It reports counts, materialization state, classifications, promotions, fields, rows, weak spots, Base Graph availability and ontology lens counts.

Page totals in the snapshot deliberately prefer `source_document_pages` or `structural_units.page_unit`, falling back to legacy `documents.page_count` only when needed. This prevents the old "sum every row's `page_count`" mistake.

13. Frontend DB Status

The Frontend health payload includes DB structure status. The UI can show:

- whether a Base Graph exists
- how many ontology lenses are present
- whether the LLM/config is ready

Base Graph readiness is derived from `source_documents` / `source_document_pages` or structural `base_unit` / `page_unit` rows. Ontology lens count comes from ontology read repository queries.

This is a small UI feature, but it matters: users should not have to ask an agent whether the DB has a Base Graph or lenses.

14. Safe External Inspection

Safe tools for direct DB inspection:

- DB Browser for SQLite
- DBeaver
- DataGrip
- SQLite CLI

Safe read patterns:

- Open the DB read-only when possible.
- Start with views before joining raw tables.
- Use `source_documents` for document-level grouping.
- Use `source_document_pages` for page order.
- Use `document_promotions` / `vw_document_promotions_current` for dynamic query fields.
- Use `evidence_atoms` when you need proof anchors.
- Use `document_page_images` only when you need image blobs; avoid dumping it in broad table scans.

Examples:

```
SELECT * FROM vw_source_document_surface ORDER BY source_title;
```

```
SELECT source_document_id, page_index, document_id, page_label FROM  
vw_source_document_pages ORDER BY source_document_id, page_index;
```

```
SELECT document_id, slot, display_value, confidence FROM vw_document_promotions_current  
WHERE slot = 'invoice_number';
```

```
SELECT ontology_id, ontology_name, node_id, canonical_label, status FROM  
vw_active_ontology_nodes ORDER BY canonical_label;
```


15. Mutation Rules

The safest rule is simple: Read freely. Write through owners.

Area	Safe Writer
Schema creation/migration	Corpus Builder <code>ensure_schema()</code>
Document materialization	Corpus Builder load/rebuild workflows
Semantic Release snapshot	Corpus Builder release activation
Embeddings	Corpus Builder embedding workflow
Base Graph	<code>basic_relation_mining</code>
Ontology lenses	Ontology Agent <code>sql_batch_execute</code>
Reset	Corpus Builder reset through Kernel workflow
Merge	Corpus Builder merge through Kernel workflow

Manual writes to these tables should be avoided:

- `documents`
- `document_payloads`
- `extracted_fields`
- `extracted_rows`
- `evidence_atoms`
- `slot_candidates`
- `document_promotions`
- `document_processing_state`
- `semantic_snapshots`
- `installation_state`
- `document_page_images`
- `embedding_chunks`
- `embeddings`

The Ontology Agent is allowed to write ontology-layer tables through its preflight/validation path. It should not be used as a broad DB surgery tool.

16. Reset, Cleanup And Merge

Reset is schema-preserving and release-preserving. It clears materialized content tables, preserves the active release snapshot, vacuums/checkpoints and returns proof that the DB is empty and the release is still present.

Scoped pipeline cleanup removes specified document-linked rows for reingest. It is not a general-purpose delete tool.

Additive merge copies SQL rows, artifacts, release state, Base Graph and ontology rows into a target DB. It must preserve materialization references and write ID maps/receipts. Merge code treats `document_page_images` as part of the document-linked table set and includes ontology support tables in the merge path.

17. Debugging Checklist

When inspecting a Corpus DB, use this order:

1. Check `installation_state` for schema and active release identity.
2. Count non-archived `documents`.
3. Check source identity fields in `documents`.
4. Check `source_documents` and `source_document_pages` for Base Graph presence.
5. Check `source_document_classifications` for `ambiguous` or `unresolved` source-level classifications.
6. Check `document_promotions` and `evidence_atoms` for queryable values and evidence.
7. Check `document_page_images` for DB-local visual evidence.
8. Check `embedding_chunks` and `embeddings` if semantic search behaves weakly.
9. Check `ontology_lenses` and `ontology_activation` if answers should reflect lenses.
10. Use read-surface views to compare base, source-document and ontology state.

18. Test Anchors

Representative regression anchors:

- 05 - Corpus Builder/dev-tests/tests/database_schema_cases.py
- 05 - Corpus Builder/dev-tests/tests/database_schema_migration_cases.py
- 05 - Corpus Builder/dev-tests/tests/test_database_page_images.py
- 05 - Corpus Builder/dev-tests/tests/test_corpus_admin.py
- 05 - Corpus Builder/dev-tests/tests/test_corpus_db_provisioning.py
- 05 - Corpus Builder/dev-tests/tests/test_search_blob_free.py
- 05 - Corpus Builder/dev-tests/tests/basic_relation_grouping_cases.py
- 05 - Corpus Builder/dev-tests/tests/basic_relation_sequence_cases.py
- 05 - Corpus Builder/dev-tests/tests/basic_relation_classification_cases.py
- 05 - Corpus Builder/dev-tests/tests/basic_relation_structural_cases.py
- Client Frontend/dev-tests/tests/ontology-agent-sql-batch-preflight.test.js
- Client Frontend/dev-tests/tests/ontology-agent-sql-batch-rollback.test.js
- Client Frontend/dev-tests/tests/ontology-agent-sql-batch-success.test.js
- 08 - Semantic Control Kernel/dev-tests/tests/
phase21_ontology_validation_fail_cases.py

19. Handover Rule

A handover-ready Corpus DB is not just a SQLite file that opens. It must be understandable as a materialized semantic artifact:

- active release identity is visible
- page-level records are source-identifiable
- Base Graph is present or explicitly absent
- page images are available in the DB and artifacts
- evidence atoms and promotions explain where facts came from
- ontology lenses are readable as overlays, not confused with base facts
- read-surface views give agents stable entry points
- manual mutation boundaries are respected

If those are true, the DB is not just filled. It is explainable.